Sensorless PMSM Drive — Theory, Observer Design, and Parameters

This document is the standalone technical reference for the simulation built across `01_pmsm_machine_and_controller.ipynb`, `02_pmsm_with_parallel_observer.ipynb`, and `03_pmsm_sensorless_control.ipynb`. It collects the machine model, the current-control design, the sensorless position/speed observer, the exact gain formulas used, and the full parameter set — with the reasoning behind each choice — in one place.

Contents

---

1. System overview

The plant is a permanent-magnet synchronous motor (PMSM) driving a fan, which loads it with a torque that grows with the square of speed. The motor is torque-controlled: an external torque reference `Tref(t)` steps through the schedule `[0, 10, 30, 50, 70, 10, 70, -20, 50]` N·m, holding each level until the drive settles before moving to the next.

The simulation is built in three stages, each adding exactly one piece so that the specific cost of each addition is isolated:

Stage	Notebook	Controller feedback	Observer
1	`01_...ipynb`	plant's own real `θr`, `ωmech`	none
2	`02_...ipynb`	still real `θr`, `ωmech`	runs in parallel, **passively monitored**
3	`03_...ipynb`	observer's estimated `θ̂`, `ω̂`	**closes the loop** (fully sensorless)

Three physical/mathematical pieces make up the whole system:

- a **dq-frame machine model** (Section 2) — the "real" plant, always evaluated with its own true rotor angle `θr`, regardless of what the controller believes;
- **dq PI current controllers with EMF feedforward** (Section 4) — the torque-mode control loop, tuned against an assumed inverter response delay;
- an **αβ-frame back-EMF observer + PLL** (Section 5) — estimates `θ̂, ω̂` from measurable stator currents and applied voltage alone, with no mechanical sensor.

2. Machine model (dq frame)

2.1 Convention: power-invariant Park transform, motor sign convention

The machine-model structure is adapted from the source reference document "RL_Modelo PMSM.pdf", which derives a PMSM model for a **generator**, using the **power-invariant** Blondel-Park transform:

```
T_BP(θ) = sqrt(2/3) * [ cosθ            -sinθ           1/sqrt(2) ]
                       [ cos(θ-2π/3)    -sin(θ-2π/3)    1/sqrt(2) ]
                       [ cos(θ+2π/3)    -sin(θ+2π/3)    1/sqrt(2) ]
```

(orthogonal, so `T_BP⁻¹ = T_BPᵀ`). Two adjustments were made to use this source for our motor:

(a) Sign convention. The source's stator KVL is written for current flowing *out* of the machine (generator reference): `u = -R·i - L·di/dt + e`. Substituting `i_motor = -i_gen` converts this to the standard motor KVL, current flowing *in*: `u = R·i + L·di/dt + e`.

(b) Where the power-invariant factor shows up. Because the transform is power-invariant rather than the more common amplitude-invariant (conventional `2/3`) transform, a factor of `√(3/2)` appears explicitly wherever the permanent-magnet flux `ψm` couples into an equation — voltage, torque, and the αβ back-EMF used by the observer — while `Rs`, `Ld`, `Lq` and the dq↔αβ relationship (a plain rotation, since discrete phases are never modeled here) are unaffected. This was **verified by power balance** (not just carried over from the source), see §2.3.

2.2 Electrical and torque equations

With `ωr = p·ωmech` the electrical angular speed and `θr` the electrical rotor angle:

```
usd = Rs·isd + Ld·disd/dt − ωr·Lq·isq
usq = Rs·isq + Lq·disq/dt + ωr·Ld·isd + √(3/2)·ωr·ψm

Te  = p·√(3/2)·ψm·isq + p·(Ld−Lq)·isd·isq
```

Define `ψm,pi = √(3/2)·ψm` (the power-invariant flux constant, §6) so these read more simply:

```
usq = Rs·isq + Lq·disq/dt + ωr·Ld·isd + ωr·ψm,pi
Te  = p·ψm,pi·isq + p·(Ld−Lq)·isd·isq
```

2.3 Power-balance verification

Multiplying the voltage equations by their respective currents and summing:

```
P = usd·isd + usq·isq
  = Rs·(isd² + isq²)                              (copper loss)
  + d/dt[ ½(Ld·isd² + Lq·isq²) ]                  (stored magnetic energy rate)
  + ωr·[ (Ld−Lq)·isd·isq + ψm,pi·isq ]            (electromagnetic power)
```

The bracketed electromagnetic-power term, divided by `p` (i.e. multiplied by `ωr/p = ωmech`), equals `Te·ωmech` exactly — confirming the equations (and the placement of `√(3/2)`) are self-consistent: `P = I²R loss + rate of stored energy + mechanical power out`. Because the transform is power-invariant, `P` needs no extra `3/2` scaling factor (unlike the conventional transform, where `P = (3/2)(usd·isd + usq·isq)`).

2.4 Reference-frame relations (used by the observer/controller, Section 5)

Since discrete phases (abc) are never modeled, the only relation needed between the dq frame and the stationary αβ frame is a plain rotation by the (real or estimated) angle:

```
i_α = id·cosθ − iq·sinθ         i_β = id·sinθ + iq·cosθ        (dq → αβ)
i_d = i_α·cosθ + i_β·sinθ       i_q = −i_α·sinθ + i_β·cosθ      (αβ → dq)
```

(and identically for voltage). Ground-truth αβ back-EMF, used for validation:

```
e_α = √(3/2)·ωr·ψm·cosθr = ωr·ψm,pi·cosθr
e_β = ωr·ψm,pi·sinθr
```

3. Mechanical model and load

```
J·dωmech/dt = Te − T_load(ωmech) − B·ωmech        dθr/dt = ωr = p·ωmech
```

`B = 0` (no viscous friction assumed — only the fan load opposes rotation). The fan load is quadratic in speed and opposes rotation in *either* direction, continuous through zero:

```
T_load(ωmech) = k_fan · ωmech · |ωmech|
```

`k_fan` is set from the one operating point given: 11 kW at 1500 rpm. With `ωmech,rated = 1500·2π/60 ≈ 157.08 rad/s` and `T_rated = P_rated/ωmech,rated ≈ 70.03 N·m` (matching the largest value in the torque schedule — confirming the schedule's units are N·m):

```
k_fan = T_rated / ωmech,rated² ≈ 2.838×10⁻³ N·m/(rad/s)²
```

`J = 0.05 kg·m²` is a user-selected value (not given in the literature) representative of an 11 kW industrial PMSM rotor + small fan load.

A note on the `-20 N·m` schedule step: since `T_load` opposes rotation in either direction, this doesn't fail to be "balanced" — the drive simply reverses and settles at a negative-speed equilibrium `ωmech·|ωmech| = -20/k_fan ⇒ ωmech ≈ -83.9 rad/s`, where both `Te` and `ωmech` are negative, so mechanical power `Te·ωmech` stays positive throughout (motoring, not braking). This step was kept deliberately as a stress case — see Section 7.

4. Current control

4.1 References (id* = 0 strategy)

`Ld` and `Lq` differ by only ~2.2% (see §6), so the reluctance-torque term `p(Ld−Lq)·isd·isq` is negligible — the standard `id*=0` strategy is used:

```
isd* = 0                    isq* = Tref / (p·ψm,pi)
```

4.2 PI gains — technical-optimum tuning against the inverter delay

Per spec, gains are computed per-axis from an assumed inverter response time constant `τu = 10 μs` (the inverter itself is not simulated — this is purely a design parameter for gain sizing, a standard technique: choosing `Kp = L/(2τu)` places a zero that cancels the electrical pole `-R/L`, leaving a closed-loop time constant of `2τu`):

```
Kp_d = Ld/(2τu)      Ki_d = Rs/(2τu)
Kp_q = Lq/(2τu)      Ki_q = Rs/(2τu)
```

Calculated values used in the notebooks, with `Rs=18 mΩ`, `Ld=180 μH`, `Lq=176 μH`, `τu=10 μs` (so `2τu = 20 μs`):

```
Kp_d = 180e-6 / 20e-6 = 9.0            Ki_d = 18e-3 / 20e-6 = 900.0
Kp_q = 176e-6 / 20e-6 = 8.8            Ki_q = 18e-3 / 20e-6 = 900.0
```

4.3 Controller equations and feedforward decoupling

```
usd_cmd = Kp_d·(isd*−isd) + ∫Ki_d·(isd*−isd)dt − ω·Lq·isq
usq_cmd = Kp_q·(isq*−isq) + ∫Ki_q·(isq*−isq)dt + ω·Ld·isd + ω·ψm,pi
```

The feedforward terms are exactly the cross-coupling / back-EMF terms from the plant model (§2.2) — added *after* the PI stage, per spec, to decouple the two current loops from the speed-dependent coupling and magnet back-EMF, so that the PI only has to reject a residual first-order error. `ω` and the currents used in these expressions are the plant's real values in notebooks 01–02, and the observer's estimates in notebook 03 (Section 7).

4.4 Voltage saturation and anti-windup

`[usd_cmd, usq_cmd]` is clamped to a circular limit `|·| ≤ Vmax = 600 V` before being applied (and before being handed to the observer as `v_α,v_β`). Each axis's integrator uses a standard back-calculation anti-windup term, `Ki·error + Kp·(u_sat − u_unsat)`, so the integral state stops winding up while that axis is clipped. In practice this machine's voltage requirement is far below the limit even at rated conditions (`|u| ≈ 108 V` at 70 N·m, 1500 rpm) — the saturation is a modeling safety net, not something that binds in normal operation (confirmed in notebook 03).

5. Sensorless position/speed estimation

5.1 Why back-EMF estimation

With no mechanical sensor, `θr` and `ωmech` must be inferred from electrical quantities alone. The standard technique — used here, and described in Urbanski (2015) [see §8] — is to estimate the back-EMF vector `(e_α, e_β)` induced in the stationary frame, which by §2.4 points in the direction `(cosθr, sinθr)` scaled by `ωr·ψm,pi`, and recover the angle from it. The paper's own conclusion (confirmed independently here, Section 7) is that this works well away from zero speed, and has two specific, well-known weaknesses: near-zero speed (where the back-EMF vector itself vanishes) and, less commonly discussed, a **sign ambiguity at negative speed** (derived below).

5.2 Observer structure — Structure B (PI-corrected Luenberger)

The source paper describes three back-EMF estimation structures; **Structure B** (§3.B in the paper — a Luenberger observer whose plain proportional correction gains are each replaced by a full PI correction) is the one implemented here, using the machine's isotropic approximation `L_obs = (Ld+Lq)/2` (valid since Ld≈Lq — the paper's αβ model assumes magnetic symmetry). With `Δiα = iα,meas − îα` (and identically for β), and a single shared error-integral state per axis `zα = ∫Δiα dt` (since both the current- and back-EMF-channel PI corrections integrate the *same* current error, just with different gains):

```
dîα/dt = −(Rs/L_obs)·îα − (1/L_obs)·êα + (1/L_obs)·vα + Ki_P·Δiα + Ki_I·zα
dêα/dt = −(Ke_P·Δiα + Ke_I·zα)
dzα/dt = Δiα
```

(β-axis: identical, mirror α→β). `vα, vβ` and `iα,meas, iβ,meas` are the physically measurable applied voltage and stator current, rotated into the stationary frame — in notebook 02 via the real `θr` (no mismatch yet); in notebook 03 via the controller's own estimate `θ̂`, which is where the sensorless dynamics actually bite (Section 7).

Sign note. The minus sign on the back-EMF channel ($d\hat{e}/dt = -(...)$, opposite in sign to the current channel) looks asymmetric but is *required* for stability — this was verified directly by Routh–Hurwitz analysis of the linearized error dynamics (not simply carried over from the source paper's typesetting, which does not make the two channels' opposite polarity obvious). Reference structure (the plain 2-state Luenberger, "Structure A") has error dynamics $d(ei)/dt = -(R/L+Ki)ei - (1/L)ee$, $d(ee)/dt = -Ke \cdot ei$, giving characteristic polynomial $s^2 + (R/L+Ki)s + Ke/L$; both coefficients must be positive for stability, which requires the observed sign.

5.3 Observer gain derivation (pole placement)

> **The tuning method in this subsection is not in the source literature.** The paper defines the Structure-B equations (§5.2) but gives no gain values and no design procedure — its experiments simply state that gains were chosen, not how. The pole-placement method below (a standard linear-observer design technique, not specific to this paper) was added to get from "here is the structure" to "here are numbers to actually run," and is a genuine design choice on top of the source material.

Gains here are derived by placing the observer's own linearized error dynamics (a 3-state system per axis: current error $ei = i-\hat{i}$, back-EMF error $ee = e-\hat{e}$, and the integral state z) at a **triple real pole** $s = -\omega n$:

$$
A = \begin{bmatrix} -(R/L+Ki\_P) & -1/L & -Ki\_I \\ Ke\_P & 0 & Ke\_I \\ 1 & 0 & 0 \end{bmatrix}
$$

giving characteristic polynomial:

$$
s^3 + (R/L + Ki\_P) \cdot s^2 + (Ki\_I + Ke\_P/L) \cdot s + Ke\_I/L
$$

Matching this to $(s+\omega n)^3 = s^3 + 3\omega n \cdot s^2 + 3\omega n^2 \cdot s + \omega n^3$ gives 2 equations for 3 remaining unknowns (the s^2 and s^0 coefficients pin down Ki_P and Ke_I uniquely; the s^1 coefficient leaves one free parameter between Ki_I and Ke_P). The remaining degree of freedom is resolved by setting Ke_P to the value a plain 2-state (Structure-A) observer at the same ωn would use, which then fixes Ki_I :

$$
\begin{aligned}
Ki\_P &= 3\omega n - R/L & \qquad Ke\_P &= \omega n^2 \cdot L \\
Ki\_I &= 2\omega n^2 & \qquad Ke\_I &= \omega n^3 \cdot L
\end{aligned}
$$

Choosing ωn . The observer's own model treats e_α, e_β as a slowly-varying disturbance (the pole-placement above assumes $de/dt \approx 0$ locally), but they are actually *rotating* at the electrical frequency ωr — up to $\omega elec,rated = p \cdot \omega mech,rated \approx 1571$ rad/s at the 1500 rpm rated point. If ωn is not well above that frequency, the estimated back-EMF vector picks up a **real, speed-proportional phase lag** relative to the true one. This was found empirically while building notebook 02: an initial design choice of $\omega n = 5000$ rad/s (only ~3.2× the rated electrical frequency) produced a ~0.6 rad position error at rated speed. Sweeping ωn confirmed the error shrinks roughly as $1/\omega n$ without fully vanishing:

`ωn` (rad/s)	position error at rated speed
5,000	~0.61 rad
20,000	~0.16 rad
40,000	~0.08 rad
60,000	~0.056 rad
80,000 (final)	**~0.043 rad**
100,000	~0.035 rad

`ωn = 80,000 rad/s` (~51× the rated electrical frequency) was chosen as the practical balance — still well-conditioned for `scipy.integrate.solve_ivp`, and comfortably resolves the fast dynamics without an excessive step-size penalty.

Calculated values used in the notebooks, with `ωn = 80,000 rad/s`, `R = Rs = 18 mΩ`, `L = L_obs = (Ld+Lq)/2 = 178 µH` (so `R/L ≈ 101.12 rad/s`):

```
Ki_P = 3×80,000 − 101.12       ≈ 239,899
Ke_P = 80,000² × 178e-6        = 1,139,200
Ki_I = 2×80,000²               = 12,800,000,000   (1.28×10¹⁰)
Ke_I = 80,000³ × 178e-6        ≈ 91,136,000,000   (≈9.114×10¹⁰)
```

The residual ~0.04 rad bias that remains even here is **not further tuning error** — it is traced to the observer's isotropic `L_obs` approximation of the real, slightly salient (`Ld≠Lq`) machine, which acts as a disturbance rotating at the electrical frequency; pure P+I correction reduces this (as gain/bandwidth increases) but cannot fully cancel a disturbance at a specific nonzero frequency — a standard sinusoidal-disturbance-rejection limit, not a bug.

5.4 Position/speed extraction — type-2 PLL

> **This entire subsection is not in the source literature.** The paper (Urbanski 2015) gives only two position/speed outputs: `sin(θ̂)=−êα/|ê|, cos(θ̂)=êβ/|ê|` (its eq. 9, a direct, unfiltered trigonometric read-off — no dynamics), and `ω̂=|ê|/ke` (its eq. 11, dividing the back-EMF magnitude by a scaling constant) — and then **shows its own eq. 11 to be unreliable** (noisy/non-sinusoidal at low speed; its Figs. 6, 7, 12), without proposing a fix. The PLL below is a standard technique from general sensorless-AC-drive practice, added here to give a usable `ω̂` given that our torque schedule crosses zero speed — exactly where the paper's own method is weakest. It is a genuine addition on top of the source material, not a derivation from it, and is called out as such.

The raw angle is still recovered the paper's way — from the estimated back-EMF vector's direction (its eq. 9, rewritten with `atan2`):

```
θraw = atan2(−êα, êβ)
```

PLL structure. A type-2 PLL is a 3-block closed loop, directly analogous to a hardware phase-locked loop (the same structure used e.g. for grid synchronization in inverters):

```
θraw ──▶ (Σ) ──▶ [phase detector: sin(θraw − θ_pll)] ──▶ [PI loop filter: Kp_pll + Ki_pll/s] ──▶ ω_pll ──▶
          ▲
          └──────────────────────────────────── θ_pll feedback ──────────────────────────────────────────────┘
```

- The **phase detector** is `sin(θraw − θ_pll)` rather than a plain subtraction — a standard, wrap-safe error signal (behaves like `θraw−θ_pll` for small errors, stays bounded for large ones, and is insensitive to the `2π` ambiguity of angles).
- The **PI loop filter** (`Kp_pll`, `Ki_pll`) is what makes it *type-2*: the integral term means `θ_pll` tracks a constant-speed input (a ramp in phase) with **zero steady-state phase error**, not just zero-error-at-DC like a plain proportional (type-1) tracker would.
- The **integrator** turns the filter's output `ω_pll` (the speed estimate) into the tracked phase `θ_pll`, closing the loop back into the phase detector — this integrator is the PLL's analog of a voltage-controlled oscillator (VCO) in a hardware PLL.

```
Kp_pll = 2·ζ·ωn,pll        Ki_pll = ωn,pll²

θ̇_pll = ω_pll + Kp_pll·sin(θraw − θ_pll)
ω̇_pll = Ki_pll·sin(θraw − θ_pll)
```

`ωn,pll = 800 rad/s` and `ζ = 0.707`. `θ̂ = θ_pll`, `ω̂ = ω_pll` (electrical); `ω̂mech = ω_pll/p`.

Choosing `ωn,pll`. Unlike the observer (§5.3), where bandwidth vs. electrical frequency directly sets steady-state accuracy, a type-2 PLL tracks a constant-speed (ramp-phase) input with **zero steady-state error regardless of bandwidth**, given enough time to settle — this was confirmed empirically: sweeping `ωn,pll` from 500 to 15,000 rad/s (with the observer fixed at `ωn=80,000`) gave an *identical* 0.0426 rad steady-state position error every time. So a high `ωn,pll` buys nothing for accuracy. It does actively hurt robustness: since the phase detector is nonlinear (`sin(·)`), a high loop gain reacting to a *large, sudden* phase step — exactly what happens during the `θ+π` flip discussed in Section 7.2 — overreacts in a cycle-slip-like way. Feeding the PLL alone a synthetic `π` phase step at the rated electrical frequency and measuring the peak speed-estimate overshoot:

`ωn,pll` (rad/s)	peak overshoot on a π phase step
600 – 1,000	~0 rad/s
1,200	~1 rad/s
1,400	~44 rad/s
1,600	~1,450 rad/s
15,000 (an earlier, over-tuned choice)	~18,900 rad/s

There is a sharp instability threshold right around the rated electrical frequency (~1,571 rad/s). `ωn,pll = 800 rad/s` was chosen for comfortable margin below that threshold, while its settling time (`~5/(ζωn) ≈ 9 ms`) remains far faster than the ~100 ms mechanical dynamics — no accuracy cost, and a much gentler reaction to the θ+π flip (see notebook 03).

Calculated values used in the notebooks, with `ωn,pll = 800 rad/s`, `ζ = 0.707`:

```
Kp_pll = 2 × 0.707 × 800 = 1,131.2
Ki_pll = 800²             = 640,000
```

6. Model and gain parameters (reference table)

6.1 Machine and load parameters

Symbol	Value	Source
`Rs`	18 mΩ	machine datasheet
`Ld`	180 μH	machine datasheet
`Lq`	176 μH	machine datasheet
`ψm` (raw)	0.053 Wb	machine datasheet
`ψm,pi = √(3/2)·ψm`	≈0.064911 Wb	derived (§2.1)
`p` (pole pairs)	10	machine datasheet
`J`	0.05 kg·m²	user-selected (not in literature)
`B`	0	assumption — only the fan load damps rotation
Rated point	11 kW @ 1500 rpm	user spec
`ωmech,rated`	≈157.080 rad/s	derived
`welec,rated = p·ωmech,rated`	≈1570.796 rad/s	derived
`T_rated`	≈70.028 N·m	derived
`k_fan`	≈2.838×10^{-3} N·m/(rad/s)^2	derived (§3)

6.2 Current-loop design

Symbol	Value
`τu`	10 µs
`Kp_d`	9.0
`Ki_d`	900.0
`Kp_q`	8.8
`Ki_q`	900.0
`Vmax`	600 V

6.3 Observer (Structure B) and PLL

Symbol	Value
`L_obs = (Ld+Lq)/2`	178 µH
`ωn` (observer)	80,000 rad/s
`Ki_P`	≈239,899
`Ki_I`	1.28×10^{10}
`Ke_P`	1,139,200
`Ke_I`	$\approx 9.114\times10^{10}$
`ωn,pll`	800 rad/s
`ζ` (PLL damping)	0.707
`Kp_pll`	1,131.2
`Ki_pll`	640,000

All of the above are computed by formula in `pmsm_common.py` (`current_pi_gains`, `observer_gains`, `pll_gains`) from the handful of top-level constants — not hardcoded twice — so the derivations above and the executed notebooks are guaranteed consistent.

7. Known limitations and observed behavior

Two distinct weaknesses of pure back-EMF-angle estimation are visible in this build — both expected from the literature, documented rather than engineered around:

7.1 Near-zero speed

The back-EMF vector magnitude is `|e| = ωr·ψm,pi`, which is small or zero near standstill — so `θraw = atan2(-êα, êβ)` carries little to no information there. This is the low-speed weakness the source paper is centrally about (its own conclusion: back-EMF methods are unreliable "for speed range below single revolution per second"). It affects the very start of the run (drive begins at rest) and the moment the drive passes through zero speed during the `-20 N·m` reversal.

7.2 Sign ambiguity at negative speed (found while building this project, not initially expected)

This one is exact and deterministic, not just a low-magnitude noise problem, and follows directly from §2.4: `(e_α,e_β) = ωr·ψm,pi·(cosθr, sinθr)`. For `ωr<0`, this is a *negative* scalar times `(cosθr,sinθr)` — indistinguishable, from the vector alone, from a *positive* magnitude at angle `θr+π`. So:

```
θraw = atan2(−êα, êβ) = θr + π      whenever ωr < 0
```

deterministically, for as long as the speed stays negative — not a transient loss of lock. This was confirmed exactly in notebook 02 (passive observer: position error sits at a flat `π` throughout the `-20 N·m` level, recovering the instant speed goes positive again) and has real consequences once notebook 03 lets the controller act on it:

- **During the reversal**: the controller's estimated currents come out negated relative to the real ones (`Park(θ̂)` at `θ̂≈θr+π` flips both axes), so it chases the wrong sign of error — producing a violent, sustained oscillation (`Te` swings to roughly ±220 N·m, over 3× rated torque) rather than a quiet mistracking, and driving the adaptive ODE solver into a much smaller step size (chaotic dynamics, not merely a slow transient).
- **After the reversal**: the drive does **not** recover on its own. `θ̂ = θr+π` turns out to be a genuine, self-consistent equilibrium of the closed loop: with the frame permanently flipped, the controller's estimated `iq` is `−iq,real`, so driving that toward `+iq*` drives the real `iq` toward `−iq*` — a commanded `+50 N·m` settles into a stable response that looks like `−50 N·m` was commanded instead (confirmed in notebook 03: `Te ≈ −50 N·m` at the end of the run, `ω̂` tracking `ωmech` accurately in *rate* the whole time, just phase-shifted by exactly `π`). Nothing later in the schedule dislodges it.

Real sensorless drives resolve this with an independent sign-of-speed reference (e.g. from the commanded `iq`, or a remembered pre-reversal state) or a re-synchronization/realignment procedure — genuinely useful future work, but out of scope for this project, which is specifically about the plain back-EMF-observer structure from the source literature.

8. References

- "RL_Modelo PMSM.pdf" (internal reference document) — dq machine model derivation (generator, power-invariant transform), re-derived to motor convention in Section 2.
- "RL_Parametros_PMSM.png" (internal reference document) — machine parameters (Section 6.1).
- Urbanski, K., "*Estimation of Back EMF for PMSM at Low Speed Range*", MM Science Journal, March 2015 — αβ back-EMF observer structures (Structure A/B), Section 5.

- `pmsm_common.py` (in this repository) — the single source of truth for every constant and gain formula quoted above.